

Rapport Labo 1

Kenan Augsburg

26 September 2025

Contents

1. Submission	3
2. Installation and Launch	4
3. Main App Description	6
4. Creating Groups	7
5. Main App Development	10
6. Watcher App	12

1. Submission

The setup script is written in fish.

To run, simply call `fish setup.fish`. Multiple tools are required for the script to run properly.

- `fish` to execute the script
- `grep` or `ripgrep` to extract the root token
- `gum` for user input
- `docker` to start the development server
- `openbao` since its cli is used to interact with the server

2. Installation and Launch

? What are tokens used for and how do they look like in OpenBao?

Tokens are used to authenticate when accessing the vault. Tokens can be used directly or generated dynamically through auth methods. (Using configured authentication providers like github OAuth)

The example provided in the documentation for token formats is the following

```
1 b.n6keuKu5Q6pXhaIcfnC9cFNd
2 r.JaKnR2AIHNk3fC4SGyyyDVoQ90
3 s.raPGTZdARXdY0KvHcWSpp5wWZIHNT
```

Plain Text

Where each token is prefixed according to their type.

- s. : Service tokens
- b. : Batch tokens
- r. : Recovery tokens

A token's body is a random Base62 value of 24 characters.

[Tokens documentation](#)^o

i Note

Root tokens are tokens with the root policy attached. Those tokens provide access to anything within OpenBao and can be set to never expire without any renewal required.

Those tokens are hard to create and should only be used for the initial setup or emergencies. They should be revoked otherwise.

1. The initial root token is generated at `bao operator init` time and has no expiration
2. Root tokens can create other root tokens. Tokens with an expiration cannot create a token without expiration.
3. Using `bao operator generate-root` with permission of a quorum of unseal key holders. (Similar to the key ceremony I guess)

i Environment setup

```
1 docker compose up -d
2 set -x BAO_ADDR http://localhost:8200
3 # Login prompt, use a token when prompted
4 bao login
5 bao status
```

Fish

? How do you set up what are the capabilities of a token?

Capabilities are a property of policies. With policies you can define what can be done based on a path. Once you have a policy you can apply the policy to a token.

For example, you can create the policy `foo`.

```
1 # file: foo.hcl
2 # Allow read of anything within secret/foo
3 path "secret/foo/*" {
4     capabilities = ["read"]
5 }
6
7 # Allow read and update for secret/bar
8 path "secret/bar" {
9     capabilities = ["read", "update"]
10 }
```

 Terraform

```
1 bao policy write foo foo.hcl
```

 Fish

And create a token with the new policy

```
1 bao token create -display-name foo -policy foo
2 # Key Value
3 # ---
4 # token s.GHmDKZEh0TKGJQ6I5McAvoy9
5 # token_accessor v0EJuFUt81Hy6J7q3YE74xJh
6 # token_duration 768h
7 # token_renewable true
8 # token_policies ["default" "foo"]
9 # identity_policies []
10 # policies ["default" "foo"]
```

 Fish

[Policy syntax](#)^o

? What is sealing and unsealing and what is it used for?

Sealing means that the server will throw away the root key it was using to decipher the secrets. This means that an intruder cannot extract the key from a memory dump but it also means that the service won't be available anymore.

Unsealing is the process of deciphering the root key. This requires the use of unsealing keys which are shared between multiple people using Shamir's secret sharing. (Similar to the key ceremony for the root CAs.)

As stated, the goal of sealing is to ensure that secrets won't leak if signs of intrusion are detected.

3. Main App Description



Why isn't it a great idea to store the ciphertexts on Vault? (We do it to avoid having to setup a database).

The vault's goal is to be a place to securely store secrets. As such it's optimized for security before performances. Accessing each message and metadata in the vault is done through the http(s) api and it doesn't seem like you can read in batch. This means that one message results in one http call (which will in turn slow down the vault as the number users/messages grows).

And more importantly, each message is stored as a ciphertext which means it's already encrypted. Storing it in the vault means that there are two layers of encryption which increases processing time without any meaningful security gains.



Quick setup

I created a `fish` script to quickly setup the dev server with the kv and transit engines

```
1 ./setup.fish
```



4. Creating Groups

? What is key rotation and what is it used for?

A key rotation consists of changing the key used for encryption. Once rotated, the old key isn't used to encrypt anymore but it can still be used to decrypt ciphertexts that were encrypted with it.

The rotation is helpful against cryptanalysis as well as potential breaches. The likelihood of cryptanalysis to work scales with the number of known ciphertexts. By rotating the key we limit the number of ciphertexts which in turn reduces the probability of cryptanalysis to work.

If the key is compromised, at least the affected data is limited to the period between key rotations and not the whole database.

? How does Vault make sure one can decrypt an old message when keys are rotated?

The key is kept. Each ciphertext has metadata attached to it which contains all the informations necessary to determine which version (rotation) of the key should be used when deciphering

? What do you think of having a key rotation every 1h?

This seems aggressive. The documentation mentions the NIST rotation guidance which tells us that the rotation should happen before a number of encryptions have been made with a key. This number of encryptions depends on the algorithm of the key but in their example a rotation every 3 month with 40M operations per day is sufficient.

Our application won't come anywhere close to that so having 1h rotation is only good to prove that the rotation works but for a production application it will just generate a lot of keys which could potentially reduce performances.

? What capabilities did you give for the transit service and why? Provide the policy file in your report

`update` . The policies documentation states `POST` and `PUT` methods are used for `create` and `update` as well as the fact that most of the project doesn't distinguish between the two and require both.

When looking at the transit API for encrypt, it's one of the functionalities which distinguish between the two. With `update` only, the encryption will fail if the key wasn't created. On the other hand, with `create` , the key will be generated with default values (for parameters.) Since we want specific types for the keys, it's better to only use `update` .



What capabilities did you give for the kv service and why? Provide the policy file in your report.

`create`, `update`, `read` and `list`. The instructions stated that users should be able to read and write to the kv (under their respective groups.)

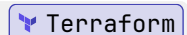
Whether they should be allowed to `update` or `delete` is ambiguous. Since it was asked that the watcher should be able to know the sender of a message, I had to give the right to `update` secrets as well as `create` and `update` their metadata.

This was done because the kv service doesn't provide any way to know which user created a secret and storing the sender inside the secret require more requests and permissions for the watcher.

Furthermore, I couldn't find any API that would let me create a secret and specify custom metadatas at once. The solution was to either `create` the secret and `update` the metadata or `create` the metadata and `update` the secret after. I chose the latter and kept the `update` permission on metadata since removing it wouldn't provide more security and the implementation in the main application already used methods that handled creation and update the same way. (The authenticity is already problematic since the client chooses the value for the sender metadata)

The `list` permission was added to make it easier for users to find messages in their groups. (Though, `scan` might have been better since users are allowed to nest their messages)

```
1 # Financial.hcl
2 # Allow encryption using the Financial key
3 path "transit/encrypt/Financial" {
4   capabilities = ["update"]
5 }
6
7 # Allow decryption using the Financial key
8 path "transit/decrypt/Financial" {
9   capabilities = ["update"]
10 }
11
12 # Allow read and write access to the Financial store
13 path "kv-v2/data/ciphertexts/Financial/*" {
14   capabilities = ["create", "update", "read"]
15 }
16
17 path "kv-v2/metadata/ciphertexts/Financial/*" {
18   capabilities = ["create", "update"]
19 }
20
21 path "kv-v2/metadata/ciphertexts/Financial/*" {
22   capabilities = ["list"]
```




```
23 }
```

```
1  # IT.hcl
2  # Allow encryption using the IT key
3  path "transit/encrypt/IT" {
4    capabilities = ["update"]
5  }
6
7  # Allow decryption using the IT key
8  path "transit/decrypt/IT" {
9    capabilities = ["update"]
10 }
11
12 # Allow read and write access to the IT store
13 path "kv-v2/data/ciphertexts/IT/*" {
14   capabilities = ["create", "update", "read"]
15 }
16
17 path "kv-v2/metadata/ciphertexts/IT/*" {
18   capabilities = ["create", "update"]
19 }
20
21 path "kv-v2/metadata/ciphertexts/IT/*" {
22   capabilities = ["list"]
23 }
```

 Terraform

```
1  # Watcher.hcl
2  # Allows listing of all the keys for Financial and IT
3  path "kv-v2/metadata/ciphertexts/Financial/*" {
4    capabilities = ["list"]
5  }
6
7  path "kv-v2/metadata/ciphertexts/IT/*" {
8    capabilities = ["list"]
9  }
10
11 # Allow access to the keys and metadata all at once
12 path "kv-v2/detailed-metadata/ciphertexts/Financial/*" {
13   capabilities = ["list"]
14 }
15
16 path "kv-v2/detailed-metadata/ciphertexts/IT/*" {
17   capabilities = ["list"]
18 }
```

 Terraform

5. Main App Development

? Show in your report that bob cannot send or receive data from the finance group.

```
nvim main.py ~/g/H/C/L/messaging
11 uv run main.py ~/g/H/C/L/messaging
18
9 mon@susanoo in repo: lab_01/messaging on P lab_01 [!] is v0.1.0 via v3.13.7 (messaging) as took 1m22s
8 A uv run main.py
7 ? Username bob
6 ? Password ****
5 ? Vault url http://localhost:8200
4 ? Group name Financial
3 ? What do you want to do? send a message
2 ? Title (The title is stored as plaintext and acts as an id) demo
1 ? Content (Finish with 'Alt+Enter' or 'Esc then Enter')
58 > This message will be refused
1 - Bob
2 Error: 1 error occurred:
3 * permission denied
4
5
6 ? What do you want to do? receive a message
7 Error: 1 error occurred:
8 * permission denied
9
10
11 ? What do you want to do? (Use arrow keys)
12 » send a message
13   receive a message
14   change group
15   quit
16
```

```
nvim main.py ~/g/H/C/L/messaging
37 ~/g/H/C/L/messaging
36
35 mon@susanoo in repo: lab_01/messaging on P lab_01 [!+] is v0.1.0 via v3.13.7 (messaging) as took 0s
34 A bao login -method=userpass username=bob
33 Password (will be hidden):
32 Success! You are now authenticated. The token information displayed below is
31 already stored in the token helper. You do NOT need to run "bao login" again.
30 Future OpenBao requests will automatically use this token.
29
28 Key Value
27 ----
26 token s.bNcA3x3jawxEu5sM5sKg3F7L
25 token_accessor UpMFvWnbTyjq6k4uXTHamfJ0
24 token_duration 768h
23 token_renewable true
22 token_policies ["default" "it"]
21 identity_policies []
20 policies ["default" "it"]
19 token_meta_username bob
18
17 mon@susanoo in repo: lab_01/messaging on P lab_01 [!+] is v0.1.0 via v3.13.7 (messaging) as took 6s
16 A bao token capabilities s.bNcA3x3jawxEu5sM5sKg3F7L "kv-v2/data/ciphertexts/IT/*"
15 create, read
14
13 mon@susanoo in repo: lab_01/messaging on P lab_01 [!+] is v0.1.0 via v3.13.7 (messaging) as took 0s
12 A bao token capabilities s.bNcA3x3jawxEu5sM5sKg3F7L "kv-v2/data/ciphertexts/Financial/*"
11 deny
10
9 mon@susanoo in repo: lab_01/messaging on P lab_01 [!+] is v0.1.0 via v3.13.7 (messaging) as took 0s
8 A bao token capabilities s.bNcA3x3jawxEu5sM5sKg3F7L "transit/encrypt/Financial"
7 deny
6
5 mon@susanoo in repo: lab_01/messaging on P lab_01 [!+] is v0.1.0 via v3.13.7 (messaging) as took 0s
4 A bao token capabilities s.bNcA3x3jawxEu5sM5sKg3F7L "transit/encrypt/Decrypt"
3 deny
```

? Show in your report that your app is working properly.

```
nvim main.py ~/g/H/C/L/messaging
11 uv run main.py ~/g/H/C/L/messaging SCROLL: 0/439 | PIN [ ]
10
9 mon@susanoo in repo: lab_01/messaging on P lab_01 [!+] is v0.1.0 via v3.13.7 (messaging) as took 3m15s
8 A uv run main.py
7 ? Username bob
6 ? Password ****
5 ? Vault url http://localhost:8200
4 ? Group name IT
3 ? What do you want to do? send a message
2 ? Title (The title is stored as plaintext and acts as an id) demo
1 ? Content (Finish with 'Alt+Enter' or 'Esc then Enter')
58 > This message will be saved
1 - Bob
2 ? What do you want to do? receive a message
3 ? Title demo
4 This message will be saved
5 - Bob
6 ? What do you want to do? (Use arrow keys)
7 » send a message
8 receive a message
9 change group
10 quit
11
```

```
nvim main.py ~/g/H/C/L/messaging
37 ~/g/H/C/L/messaging SCROLL: 0/1062 | PIN [ ]
36
35 mon@susanoo in repo: lab_01/messaging on P lab_01 [!+] is v0.1.0 via v3.13.7 (messaging) as took 0s
34 A bao login -method=userpass username=bob
33 Password (will be hidden):
32 Success! You are now authenticated. The token information displayed below is
31 already stored in the token helper. You do NOT need to run "bao login" again.
30 Future OpenBao requests will automatically use this token.
29
28 Key Value
27 ---
26 token s.bt16CnQzcrFn7zfJiswMygPD
25 token_accessor 1YLpFvBkQDf43rM9DsNxLjxr
24 token_duration 768h
23 token_renewable true
22 token_policies ["default" "it"]
21 identity_policies []
20 policies ["default" "it"]
19 token_meta_username bob
18
17 mon@susanoo in repo: lab_01/messaging on P lab_01 [!+] is v0.1.0 via v3.13.7 (messaging) as took 1s
16 A bao token capabilities s.bt16CnQzcrFn7zfJiswMygPD "kv-v2/data/ciphertexts/IT/*"
15 create, read
14
13 mon@susanoo in repo: lab_01/messaging on P lab_01 [!+] is v0.1.0 via v3.13.7 (messaging) as took 0s
12 A bao token capabilities s.bt16CnQzcrFn7zfJiswMygPD "transit/encrypt/IT"
11 update
10
9 mon@susanoo in repo: lab_01/messaging on P lab_01 [!+] is v0.1.0 via v3.13.7 (messaging) as took 0s
8 A bao token capabilities s.bt16CnQzcrFn7zfJiswMygPD "transit/decrypt/IT"
7 update
```

```
nvim main.py ~/g/H/C/L/messaging
37 ~/g/H/C/L/messaging SCROLL: 0/1319 | PIN [ ]
36
35 mon@susanoo in repo: lab_01/messaging on P lab_01 [!+] is v0.1.0 via v3.13.7 (messaging) as took 0s
34 A bao kv get kv-v2/ciphertexts/IT/demo
33 ===== Secret Path =====
32 kv-v2/data/ciphertexts/IT/demo
31
30 ===== Metadata =====
29 Key Value
28 ---
27 created_time 2025-09-24T16:28:28Z
26 custom_metadata <nil>
25 deletion_time n/a
24 destroyed false
23 version 1
22
21 ===== Data =====
20 Key Value
19 ---
18 value vault:v1:7F42SKVA6IUF6zxVAK36Zz7FWH3w4Nb8oEM0j3wDz+3JtIWLxgvf3+Y0bm0ccmrD3DlkB2cEjQdJm+pdBA==
17
16 mon@susanoo in repo: lab_01/messaging on P lab_01 [!+] is v0.1.0 via v3.13.7 (messaging) as took 0s
15 A bao write transit/decrypt/IT ciphertext="vault:v1:7F42SKVA6IUF6zxVAK36Zz7FWH3w4Nb8oEM0j3wDz+3JtIWLxgvf3+Y0bm0ccmrD3DlkB2cEjQdJm+pdBA="
14
13 Key Value
12 ---
11 plaintext Vghpc2UgbWVzc2FnZSB3aWxsIGJlIHhndmVkcj0gQm9i
10
9 mon@susanoo in repo: lab_01/messaging on P lab_01 [!+] is v0.1.0 via v3.13.7 (messaging) as took 0s
8 A echo Vghpc2UgbWVzc2FnZSB3aWxsIGJlIHhndmVkcj0gQm9i | base64 -d
7 This message will be saved
6 - Bob
```

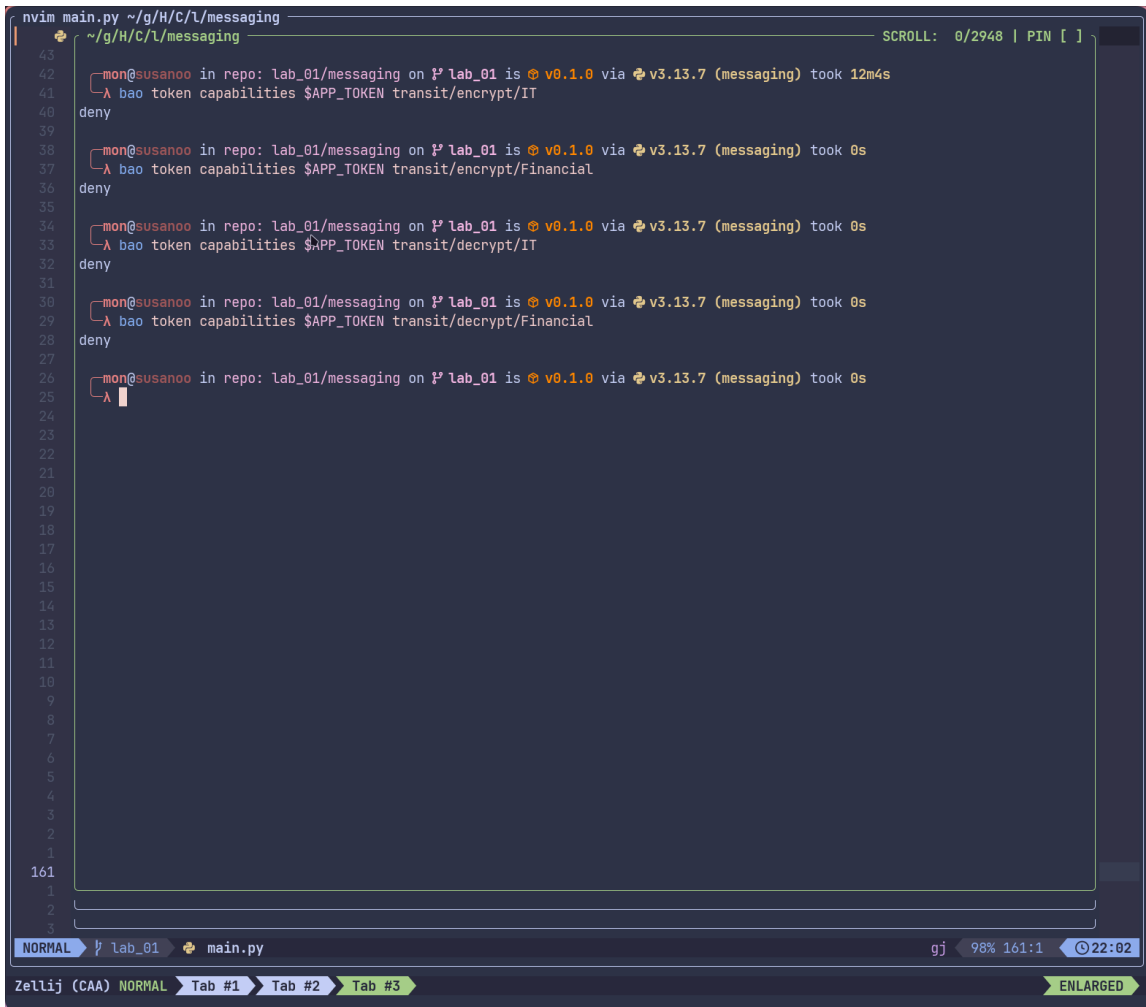
6. Watcher App

Note

To use the watcher, you need an `APP_TOKEN` which is generated during the `Watcher` step in `setup.fish`.

Running this step again doesn't break anything

Show in your report that your app token cannot encrypt or decrypt data.



```
nvim main.py ~/g/H/C/L/messaging
~/g/H/C/L/messaging  SCROLL: 0/2948 | PIN [ ]
43
42 mon@susanoo in repo: lab_01/messaging on P lab_01 is v0.1.0 via v3.13.7 (messaging) took 12m4s
41 A bao token capabilities $APP_TOKEN transit/encrypt/IT
40 deny
39
38 mon@susanoo in repo: lab_01/messaging on P lab_01 is v0.1.0 via v3.13.7 (messaging) took 0s
37 A bao token capabilities $APP_TOKEN transit/encrypt/Financial
36 deny
35
34 mon@susanoo in repo: lab_01/messaging on P lab_01 is v0.1.0 via v3.13.7 (messaging) took 0s
33 A bao token capabilities $APP_TOKEN transit/decrypt/IT
32 deny
31
30 mon@susanoo in repo: lab_01/messaging on P lab_01 is v0.1.0 via v3.13.7 (messaging) took 0s
29 A bao token capabilities $APP_TOKEN transit/decrypt/Financial
28 deny
27
26 mon@susanoo in repo: lab_01/messaging on P lab_01 is v0.1.0 via v3.13.7 (messaging) took 0s
25 A
24
23
22
21
20
19
18
17
16
15
14
13
12
11
10
9
8
7
6
5
4
3
2
1
161
1
2
3
NORMAL P lab_01 main.py gj 98% 161:1 22:02
Zellij (CAA) NORMAL Tab #1 Tab #2 Tab #3 ENLARGED
```

? Show in your report that your app is working properly

The screenshot shows a terminal window with two main sections of output:

Top Left (main.py):

```

nvim main.py ~/g/H/C/L/messaging
uv run main.py ~/g/H/C/L/messaging 0/223 | PIN [ ]
...messaging on 'lab_01 [!]' is v0.1.0 via v3.13.7 took 25s
└─ uv run main.py
? Username ceo
? Password ****
? Vault url http://localhost:8200
? Group name Financial
? What do you want to do? send a message
? Title (The title is stored as plaintext and acts as an id)
demo_message
? Content (Finish with 'Alt+Enter' or 'Esc then Enter')
> message body
? What do you want to do? change group
? Group name IT
? What do you want to do? send a message
? Title (The title is stored as plaintext and acts as an id)
demo_message_it
? Content (Finish with 'Alt+Enter' or 'Esc then Enter')
> body
? What do you want to do? (Use arrow keys)
  send a message
  » receive a message
  change group
  quit

```

Bottom Left (watcher.py):

```

uv run watcher.py ~/g/H/C/L/messaging 0/5711 | PIN [ ]
... lab_01 [!]' is v0.1.0 via v3.13.7 (messaging) took 56s
└─ uv run watcher.py
group=Financial, title=demo_message, sender=ceo
group=IT, title=demo_message_it, sender=ceo

```

Right (JSON Output):

```

e):
{
  "ciphertexts/{group}": {
    "demo_message": {
      "body": "demo_message",
      "sender": "ceo",
      "title": "demo_message"
    },
    "demo_message_it": {
      "body": "demo_message_it",
      "sender": "ceo",
      "title": "demo_message_it"
    }
  }
}

```

The terminal status bar at the bottom shows: NORMAL | lab_01 | watcher.py | f check_new_messages | Alt <Click> - group, Alt <Right-Click> - ungroup all | Zellij (CAA) NORMAL | Tab #1 | Tab #2 | Tab #3 | SPREAD

- Top left corner: The main app connected as ceo
- Bottom left: The watcher started just before the main app
- Right: All the existing messages (keys only)